Genesis OS / Relationship Scores Drift System Official Companion Contract

Merged Build-Set Addendum v1

Detailed engineering contract calibrated to the workbook-governed scoring stack, the corrected master blueprint, and the drift addendum.

Purpose	**Promote drift from a sidecar note into an official companion-system contract that developers can implement without inventing their own drift logic.**
Scope	Defines drift lanes, inputs, rollups, eligibility gates, state machine, payloads, write-backs, confidence rules, decay, shadow-activation interlock, service boundaries, API contracts, QA requirements, and rollout discipline.
Authority	This document follows the workbook and the corrected unified master blueprint. It does not replace workbook score law; it governs how longitudinal drift evidence enriches forecast, guidance, and selected medium-lane traits.
Design stance	A bad week is not a tattoo. Drift is allowed to explain recent distance and evolving behavior without rewriting slow identity on flimsy evidence.

Primary references used in this merge: the corrected unified master blueprint and the System Gap Audit + Drift Addendum.

1. Why this document exists

The current package already acknowledges drift across messaging telemetry, Sanctuary interventions, relationship forecast, companion architecture, async scans, and later-phase richer runtime logic. The real problem is not whether drift belongs in Genesis OS. It does. The problem is that the strongest deterministic drift rules were still living in the standalone DriftEligibility file and the gap-audit addendum instead of inside the official build package. That split invites implementation drift, inconsistent thresholds, and app-layer improvisation.

This contract closes that gap. It turns drift into an official companion-system module with explicit state lanes, evidence requirements, rollup metrics, gating logic, outputs, persistence rules, AI visibility discipline, and service ownership. The goal is simple: a developer should be able to build the drift system from this document and the workbook-governed package without guessing what drift means, when it may fire, or how far it is allowed to reach.

2. Non-negotiable positioning inside the Genesis OS stack

- Drift is a companion system, not the score law. The workbook still owns compatibility math, pair quantum, overlays, and chemistry.
- Drift belongs to the medium lane. It may shape pair guidance, forecast, and selected medium-lane traits, but it must not freely rewrite archetype, baseline shadow, or core blueprint outputs.

- Drift explains recent distance, pressure patterns, and trend movement. It is not a moral verdict, not a diagnosis, and not a shortcut to relabeling a user because one conversation went sideways.
- Drift has stronger influence on relationship forecast and pair guidance than on compatibility. Compatibility remains comparatively stable; forecast is where recent reality matters most.
- All drift writes must be auditable. Every derived write needs provenance, rules_version, feature freshness, and the rollup window used.
- Drift must decay. Normalized behavior should gradually reduce active drift pressure rather than preserving old turbulence forever.

3. Drift system purpose and operating definition

In Genesis OS, drift is the governed interpretation layer that measures whether recent relationship behavior suggests temporary distance, protective withdrawal, avoidant destabilization, or higher-risk terminal unraveling. It is built to answer a narrower question than compatibility: "What is happening now, and how confidently do recent events suggest it is changing?"

The drift system therefore sits between raw telemetry and forecast. It reads message cadence changes, unanswered bids, ritual drop-off, repair behavior, arousal volatility, contradiction severity, and movement stability. It converts these into a bounded drift status and a reason distribution that pair services, Sanctuary, couple tools, and AI guidance can consume without treating momentary instability as slow-trait truth.

3.1 What drift is allowed to do

- Generate governed outputs such as drift_status, drift_reason_probabilities, drift_confidence, drift_snapshot, and selected drift reason codes.
- Influence pair-state forecast, couple guidance, intervention suggestions, and caution bands.
- Write medium-lane contextual signals and selected micro-drift updates when eligibility gates are met.
- Trigger Sanctuary or repair-oriented experiences when configured thresholds are crossed.
- Feed calibration and outcome learning pipelines once telemetry and outcomes accumulate.

3.2 What drift is not allowed to do

- Overwrite archetype or baseline shadow because of one volatile window.
- Mutate the workbook compatibility engine directly in the client or ad hoc in product code.
- Expose raw hidden risk logic to a match partner.
- Let AI narrative outputs become identity truth without evidence and consent.
- Circumvent lane governance or confidence gating.

4. Lane governance and update discipline

Genesis OS already uses update lanes. This drift contract must obey them rigorously because drift is powerful enough to contaminate identity if lane boundaries get sloppy.

Lane	What lives here	Update speed	Drift permission
Fast lane	Daily or session state, emotional volatility, immediate pressure, pre/post chat shifts, active conflict posture	Can change quickly	Drift may read and write state snapshots here freely with provenance and decay.

Medium lane	Recent relationship patterning, attachment stability index, shadow pressure, pacing movement, repair trajectory, emerging pair strain	Rolling windows only	Drift is strongest here. It may update governed medium-lane indicators when gates and confidence thresholds are met.
Slow lane	Archetype, baseline shadow assignment, slow identity blueprint, durable profile truth	Rare, high-evidence only	Drift cannot rewrite slow traits unless DriftEligibility passes and multi-source validation is satisfied.

Default v1 stance from the addendum remains mandatory: keep slow drift OFF by default except micro-drift for shadow_trait_prior and attachment_stability_index, and keep archetype drift OFF until confidence is demonstrably high. That is not caution for caution's sake. It protects the whole platform from identity chaos.

5. Canonical inputs, rollups, and event sources

Drift is only as good as the event spine beneath it. The corrected build package already expects event-sourced logging, packet building, Sanctuary telemetry, and feature-store freshness. This contract specifies which of those sources are mandatory for drift and how they should be interpreted.

5.1 Mandatory input families

Input family	Examples	Why drift needs it	Minimum handling rule
Messaging telemetry	message_count, reply latency, silence gaps, unanswered bids, initiation balance, conversation days	Drift lives or dies on cadence and bid-response changes; messaging is the clearest behavioral source	Store append-only events and 20/30/50-window rollups before any drift write is allowed.
Repair telemetry	repair_attempt_count, repair_success_count, normalized repair latency, reconnection events, apology success	Separates temporary strain from terminal collapse	Repair metrics must be windowed and freshness-scored; missing repair data lowers confidence.
Contradiction / cluster pressure	cluster burden, contradiction severity, active cluster hits, pattern-risk events	Drift should not interpret behavior as stable change while the story is still internally noisy	Use versioned cluster outputs only; do not derive contradictions ad hoc.
Arousal / volatility signals	A_slope, A_std, pre/post state shifts, escalation events	Distinguishes turbulence from settled pattern change	Volatility and slope are gating inputs, not just explanatory fields.
Ritual and Sanctuary engagement	ritual drop-off, reflection completion, check-in abandonment, DM reflection responses	Reveals disengagement, shutdown, or protective pullback across self-work surfaces	Use only consented, lane-safe summaries; do not expose private Sanctuary details to a partner.
Movement stability	move_entropy, pattern	Prevents false drift writes	Windowed entropy is a

	switching, stable vs chaotic behavior paths	when behavior is just bouncing everywhere	hard gate for identity-related drift.
Current genome and profile context	current_genome, attachment_stability_index, shadow_trait_prior, readiness context	Drift needs a current baseline so it can measure movement instead of reacting blindly	Read only through governed feature-store snapshots with provenance.
Couple and relationship context	pair_id, relationship mode, breakup state, cooldown state, couple repair loop context	The same silence means different things depending on pair stage	Drift scoring must read pair stage and active relationship state before issuing guidance.

5.2 Required minimum rollup metrics to lock in code

Metric	Definition in v1	Why it exists
E50 escalation rate	Escalation-event rate across the last 50 relevant events	Captures persistent conflict pressure rather than single spikes.
E20 escalation rate	Escalation-event rate across the last 20 relevant events	Protects the engine from redefining identity mid-storm.
R50 repair success rate	Successful repair events divided by repair attempts across last 50 events	Separates strain with recovery from strain without repair.
L50 normalized repair latency	Normalized delay between rupture and repair across last 50 events	Measures whether repair exists but is too slow to stabilize the pair.
A_slope50 arousal slope	Trend slope for arousal or activation across last 50 events	Blocks drift when the trend is still actively swinging.
A_std50 arousal volatility	Standard deviation of arousal activation in the last 50 events	Catches turbulence and instability before drift writes.
C50 contradiction severity	Aggregated contradiction severity over the last 50 events/windows	Prevents identity movement while the story is internally noisy.
move_entropy50	Normalized entropy of behavioral movement across the last 50 events	Rejects drift when behavior is too chaotic to interpret as settled change.

6. DriftEligibility v1: lock-the-engine gates

The following gates come directly from the gap audit drift addendum and should be treated as canonical v1 merge rules. These are not optional suggestions. They are the safety rails that stop the system from turning medium-lane interpretation into identity vandalism.

Gate / rule	Locked threshold	Meaning	Implementation note
Purpose	Only allow slow-trait drift when stable evidence suggests the person has actually changed or onboarding was inaccurate	Prevents identity chaos and emotional whiplash	Apply this purpose test before any slow-lane mutation is even considered.

Inputs	rollup_n50, rollup_n30, current genome, config thresholds	Defines the minimum input contract	No service may compute eligibility from raw events only.
Outputs	eligible: bool, reason_codes: string[], confidence: float	Defines the mandatory response packet	Every write must retain the full eligibility packet for audit.
G0 coverage gate	At least 50 total events and at least 10 unique days covered	One bad weekend cannot redefine someone	If coverage fails, slow drift remains ineligible and only fast-state snapshots may update.
G1 recent-instability block	Block if E20 > 0.35 or A_std50 > 0.22	Do not redefine identity mid-storm	Recent instability may still inform forecast, but not slow-trait drift.
G2 settled-trend gate	abs(A_slope50) <= 0.05	Postpone drift while the trend is still swinging	Slope is a hard gate, not just a confidence reducer.
G3 contradiction-control gate	C50 <= 0.55 or improved by at least 0.10 versus prior 50-window	Blocks drift when the story is still internally noisy	Allows improving contradiction profiles to qualify even before perfection.
G4 stability gate	move_entropy_norm <= 0.78	Reject drift when behavior is chaotic	Entropy is the difference between settled change and random oscillation.
Eligibility confidence	Weighted confidence across escalation, volatility, slope, contradiction severity, and move entropy must be >= 0.60	Prevents brittle pass/fail logic	Store the subcomponents so QA can explain why confidence failed.
Default v1 stance	Keep slow drift OFF by default except micro-drift for shadow_trait_prior and attachment_stability_index; archetype drift OFF until confidence is high	Smart caution; no cowboy identity rewrites	This default remains active unless a future workbook-governed revision explicitly changes it.

7. Drift outputs and canonical payload contract

Drift must write a governed packet that downstream services can rely on. Free-form narrative-only outputs are not enough. The system needs structured fields first and explanation second.

Output field	Type	Description	Consumer services
drift_status	enum	Primary interpreted state of recent relationship movement. Suggested v1 values: stable, mild_drift, protective_drift, avoidant_drift,	Forecast service, couple tools, Sanctuary, internal reporting

		terminal_risk, recovering, suppressed, insufficient_data	
drift_reason_probabilities	object/map	Normalized probability distribution over reason families such as temporary overload, protective withdrawal, avoidance pattern, conflict fallout, disconnection, ritual drop-off, instability storm	Forecast service, AI packet builder, support tooling
drift_confidence	float 0-1	Confidence in the current drift interpretation based on coverage, stability, contradiction control, volatility, and trend-settledness	Forecast, UI suppression logic, admin review
drift_snapshot	versioned object	Timestamped summary of rollups, active gates, reason codes, and exposure-safe diagnostics	Event ledger, feature store, audit, recalculation
drift_reason_codes	string[]	Short canonical codes that explain which behaviors drove the outcome	QA, admin review, AI translation layer
drift_eligibility	object	The full eligibility packet: eligible, failed_gates, confidence, windows_used, rules_version	Audit, admin review, downstream guards
drift_decay_state	object	Tracks when pressure last increased, last normalized, and current decay trajectory	Forecast and pair guidance
drift_visibility_class	enum	internal_only, user_private, match_safe, shared_couple	Client rendering and AI packet scoping

7.1 Suggested v1 drift status semantics

- stable - no meaningful recent distance signal; drift packet may still exist with low pressure and low concern.
- mild_drift - small but real softening in cadence, follow-through, or ritual consistency; often informational rather than alarming.
- protective_drift - distance appears more self-protective or overwhelm-driven than hostile; repair paths may still be open.
- avoidant_drift - repeated withdrawal, silence, or deactivation patterns are meaningfully shaping pair safety.

- terminal_risk - persistent pressure plus low repair plus drop-off patterns strongly suggest the connection is actively degrading.
- recovering - prior drift signal is easing because normalization and repair have improved the relevant windows.
- suppressed - data exists but must not be shown because confidence, consent, or visibility policy fails.
- insufficient_data - event coverage or freshness is too weak for a real drift classification.

8. Drift state machine

The drift engine needs a strict state machine so services do not treat drift like a vibes-only label.

State	Entry condition	Exit condition	Notes
insufficient_data	Coverage or freshness below minimum thresholds	Coverage satisfied and freshness restored	No pair-facing action beyond optional soft prompt or internal wait state.
monitoring	Coverage exists but meaningful drift threshold not crossed	Transitions to active_* state when pressure or reason certainty crosses configured trigger	Default neutral runtime state.
active_mild	Low-to-moderate drift pressure with acceptable confidence	Decay to recovering/stable, or escalate to protective/avoidant	Usually user-private or match-safe only at soft label level.
active_protective	Reason profile favors overwhelm, self-protection, or non-hostile pullback	Repair normalization, time decay, or escalation into avoidant/terminal profile	Best consumer is Sanctuary/couple guidance, not public judgment.
active_avoidant	Repeated withdrawal/shutdown profile with stronger persistence	Recovery through repair and normalization, or escalation to terminal_risk	High forecast relevance.
active_terminal_risk	Persistent drift plus low repair plus drop-off suggests major degradation	Only exits through extended normalization, confirmed breakup flow, or manual admin recovery path	Usually not match-visible; primarily internal/user-private.
recovering	Earlier active drift is decaying after normalized behavior	Return to monitoring/stable when decay completes; re-escalate if pressure returns	Important for positive reinforcement and pair guidance.
suppressed	Engine computed drift but visibility/consent policy suppresses rendering	Policy or consent changes, or internal use only	Packet remains available to authorized internal services.

9. Decay, persistence, and write-back rules

Drift must decay when behavior normalizes. A drift system that only escalates becomes a memory of damage instead of a live interpretation engine.

9.1 Persistence rules

- Every drift snapshot must store created_at, last_recomputed_at, feature_freshness, rules_version, and originating rollup windows.
- Drift snapshots are append-only in the event ledger. Current active drift state is a derived view, not the only copy.
- Downstream services should read the latest valid snapshot plus the current active derived status.
- If data freshness falls below the configured window, active drift should downgrade to monitoring, suppressed, or insufficient_data rather than pretending certainty remains frozen forever.

9.2 Decay rules

- When escalation drops, reply behavior normalizes, repair success improves, volatility falls, and contradiction pressure eases, drift pressure should decay gradually rather than snap to zero unless a hard reset state is explicitly defined.
- Decay should be faster for fast-state drift than for medium-lane drift. Slow-lane implications require the most patience and the strongest evidence.
- A recovering state should be emitted before fully stable when an earlier active drift state materially existed.
- Decay must be reversible. New pressure inside the active observation window can re-escalate the state.

9.3 Write-back boundaries

Target	Allowed drift write-back	Not allowed	Reason
Forecast layer	Yes - strong influence through forecast state, caution bands, and pair guidance	Direct replacement of compatibility score law	Forecast is meant to reflect recent relational reality.
Compatibility layer	Yes, modestly and indirectly if later calibrated and explicitly governed	Ad hoc runtime score mutation in app code	Compatibility should remain comparatively stable.
Medium-lane profile traits	Yes, only for selected fields such as shadow_trait_prior or attachment_stability_index when eligibility rules pass	Unbounded profile rewriting	Allows meaningful adaptation without identity chaos.
Slow identity outputs	Only under DriftEligibility + multi-source validation + high confidence	Single-window archetype or baseline shadow overwrite	Protects the blueprint.
AI narrative packet	Yes, within consent and visibility scope	Raw hidden risk leakage to partner	Narrative is allowed to explain, not to overexpose

			engine internals.
User-facing match surfaces	Only softened translations at approved visibility classes	Raw drift reason codes or shadow activation labels	Partner safety and governance.

10. Shadow Activation interlock that must ship beside drift

The gap-audit addendum is explicit: shadow activation rules belong next to drift because they explain active pressure without rewriting baseline trait assignments. That interlock is not cosmetic. It is how the system distinguishes temporary activation from durable identity.

Shadow activation	Signature from addendum	Implementation posture
Manipulator	control + outcome steering + deflection when challenged	Keep conservative and persistence-based to avoid false labeling; use primarily as internal pressure signal and forecast contributor.
Stonewaller	withdrawal + shutdown + control-loss sensitivity	Treat as fast/medium pressure signal, not baseline rewrite; especially relevant for avoidant drift interpretation.
Love Bomber	high pursuit + reassurance hunger + intensity under low safety	Useful for forecast and couple guidance; should not auto-become public identity language.
Chameleon	appeasement + self-erasure under threat	Keep mostly internal and softly translate only for the owner where appropriate.
Scorekeeper	unfairness focus + accounting + re-litigating	Requires event labels for history references and unfairness triggers; heavy relevance for conflict-afterglow drift.
Self-Saboteur	rupture/self-punish loop + shame + pre-emptive collapse	Feed forecast heavily and compatibility modestly at first; pair guidance should prioritize repair and stabilization.

Implementation rule: active shadow signals may help explain a drift state, change reason probabilities, or increase caution, but they do not rewrite baseline shadow assignments on their own. Shadow activation is the pressure weather; baseline shadow is the slower climate.

11. Source-system integration across the official package

11.1 Messaging and telemetry

- Messaging telemetry is a primary drift source. Message count, response time, initiation balance, silence gaps, repair telemetry, reassurance events, withdrawal events, and milestones should be instrumented early because drift depends on them.
- The messaging service writes raw events to the event ledger. Drift consumes rollups from the feature store or rollup service rather than scraping UI state or client-side logs.

11.2 Sanctuary

- Sanctuary is not the drift engine, but it is a major evidence and intervention partner. Drift may read allowed summaries such as ritual drop-off, reflection abandonment, emotional-state shifts, and module completions.
- Sanctuary may consume drift outputs to decide whether to show stillness prompts, repair suggestions, reflection check-ins, or softer pacing guidance.
- Private Sanctuary content remains governed by consent and visibility class. A partner must not see private reflection details simply because drift consumed a summary.

11.3 Human Emotion Genome

- The genome layer provides current genome context and feature registries that drift can read as baseline context. Drift does not override the genome registry; it writes separate drift packets with provenance and freshness.

11.4 Forecast and readiness

- Readiness is versioned separately and should not be casually rewritten by drift.
- Forecast is the primary downstream consumer of drift. Drift should materially inform caution bands, pair-state summaries, and projected stability language.

11.5 Quantum Mirror / AI

- Quantum Mirror may narrate drift only as a governed narrative layer using approved visibility classes, evidence bands, and refusal rules.
- AI may translate drift for the user or couple context only when consent allows. It may not author new drift truths from vibes or isolated text.

12. API, packet, and storage contract

Developers need strict packets. A drift system described only in prose is how ten different services end up with eleven interpretations.

12.1 Recommended canonical objects

Object	Minimum fields	Write owner	Notes
drift_snapshot	snapshot_id, pair_id, user_id_scope if applicable, drift_status, drift_reason_probabilities, drift_reason_codes, drift_confidence, failed_gates, passed_gates, rollup_refs, feature_freshness, rules_version, scoring_version, created_at	Drift service	Append-only historical record.
drift_eligibility_result	eligibility_id, eligible, confidence, reason_codes, gates_evaluated, rollup_window_ref,	Drift service	Stored even when eligibility fails.

	created_at, rules_version		
drift_state_view	pair_id, active_drift_status, current_confidence, latest_snapshot_id, decay_state, visibility_class, last_updated_at	Derived view / read model	For client-safe and service-safe reads.
drift_reason_rollup	pair_id, reason_code, probability, contributing_features, freshness, computed_at	Drift service / feature store	Useful for explainability and audits.
shadow_activation_event	event_id, pair_id or user_id, shadow_code, activation_score, evidence_refs, computed_at, rules_version	Shadow activation service or drift companion service	Must remain distinct from baseline shadow assignment.

12.2 Suggested endpoint surface

- POST /drift/recompute - recompute current drift packet from latest eligible rollups.
- GET /pairs/{pair_id}/drift - return authorized drift state view for internal or user-private consumption depending on requester and consent.
- GET /pairs/{pair_id}/drift/history - return authorized snapshot history for admin review or owner-only reflection views.
- POST /drift/eligibility/check - run gates against a specified rollup window and return the structured eligibility result.
- POST /drift/admin/review - controlled override or annotation endpoint for support and QA, always audit-logged.
- POST /drift/decay/recompute - optional asynchronous decay recalculation if the architecture separates active recompute from passive decay.

13. Visibility, policy, and user-safe translation rules

Drift is emotionally sensitive. Even accurate drift signals can do damage if surfaced crudely.

Visibility class	Who can see it	Allowed content	Blocked content
internal_only	Authorized services, admins, QA tooling	Raw reason codes, gate failures, hidden pressure logic, activation events	Any direct end-user display
user_private	The user themself, when policy allows	Soft translations, caution language, owner-safe reflections, trend summaries	Partner-exposing private interpretations
match_safe	Both parties only where explicitly defined	Highly softened pair-safe phrases such as "recent connection strain" or "communication slowed	Raw labels like avoidant_drift, shadow activation names, hidden confidence math

		recently"	
shared_couple	Confirmed relationship/couple-mode contexts with correct consent	Couple-safe repair and stabilization guidance	Private single-user Sanctuary source detail

13.1 User-facing language stance

- Use words like signal, recent pattern, trend, pressure, strain, distance, repair window, or stabilization path.
- Do not use destiny language, diagnosis language, or certainty theatre.
- Translate hidden engine outputs into reflective, non-accusatory copy.

14. QA harness, failure modes, and rollout guidance

14.1 Mandatory QA scenarios

- Coverage failure: less than 50 total events or fewer than 10 unique days must block slow-trait drift.
- Storm block: E20 above threshold or A_std50 above threshold must prevent identity movement but still allow forecast caution.
- Swinging trend block: abs(A_slope50) above threshold must prevent eligibility.
- Improving contradiction path: C50 slightly above target but improved by at least 0.10 should be allowed under G3.
- Entropy failure: move_entropy over threshold must block settled-change interpretation.
- Normalization recovery: previously active drift should decay into recovering and then stable when relevant metrics normalize.
- Visibility suppression: internal drift packets must remain hidden from unauthorized surfaces.
- Shadow interlock: activation events must influence reason probabilities without overwriting baseline shadow.

14.2 Rollout recommendation

- Ship drift first as forecast-facing and guidance-facing before giving it any slow-trait rewrite authority.
- Keep archetype drift disabled in v1.
- Allow only micro-drift fields explicitly named in this contract or future workbook-governed revisions.
- Capture telemetry, review false positives, and calibrate coefficients before expanding drift reach.

15. Merge instruction for the official build set

This addendum should be inserted into the official Genesis OS build set as the canonical drift companion contract beside the corrected master blueprint, the operational closure spec, the missing layers blueprint, the readiness and forecast contract, the cluster rule pack specification, the Sanctuary state-machine spec, and the Human Emotion Genome registry. The standalone DriftEligibility file should no longer be treated as an orphan rule source after this merge. Its material should now be considered incorporated here.

In practical handoff terms, developers should read the workbook first for score law, the corrected master blueprint for architecture and service boundaries, and this drift contract for the longitudinal drift governor. If any implementation choice conflicts with workbook score law, the workbook wins. If drift behavior is unspecified in older docs but specified here, this document wins until formally revised.

16. Final implementation statement

Drift now has an official place in the package. It is not a floating concept, not a private founder explanation, and not something a dev should "probably just build from common sense." Build it as a governed medium-lane companion system with strict gates, auditable outputs, decay, and visibility discipline. Let it improve forecast and pair guidance. Do not let it rewrite identity recklessly. That is how it stays useful instead of becoming chaos with a dashboard.